SLE-4: Final Viva & Architecture Decision Record

02AML204 – Introduction to Artificial Intelligence

SY B.Tech. CSE (AI & ML) | SEM-VI | 40% Weightage

PRN	25UAM034	**Name**	Anju sunil koli
Division	A	**Date**	14 October 2026
Repository			

PART A — Architecture Decision Record (ADR)

ADR Title: Dual Implementation of BFS and DFS for Comparative Graph Search Analysis

Status: Accepted · Date: 14 October 2026

1. Context

Graph search is a foundational problem in Artificial Intelligence: given a start node and a goal node, the system must discover a connecting path. In SLE-1 the two classic uninformed algorithms — Breadth-First Search (BFS) and Depth-First Search (DFS) — were implemented from scratch. SLE-2 then placed both implementations under controlled measurement on one shared graph (start node = A). Three goal positions were selected to represent graded difficulty: Goal B (best-case), Goal I (average-case) and Goal K (worst-case). Wall-clock time was captured with Python's timeit module (ten independent runs per goal; minimum, average and maximum retained). Node-expansion counters were embedded directly inside each search procedure so that algorithmic work, not just elapsed time, could be compared.

Independently, SLE-3 produced a C4 architectural description (Context, Container, Component, Code). That description already isolated Input handling, the Search concern, and Output reporting as distinct components. Consequently a design decision crystallised: should the system specialise on a single algorithm, or should both BFS and DFS be retained as peer modules? This ADR records the decision, the alternatives that were weighed, the concrete evidence that justified the choice, and the consequences that follow.

2. Decision

Both BFS and DFS are retained as first-class, interchangeable search modules. They are driven by identical inputs and judged by the same pair of metrics — mean execution time in milliseconds and the number of nodes expanded. The dual-implementation stance produces an empirical, side-by-side comparison instead of a purely theoretical preference for one algorithm.

3. Alternatives Considered

- **BFS only:** Would guarantee shortest paths (fewest edges) on unweighted graphs. The drawback is the complete absence of comparative data against DFS, leaving the practical trade-offs unexplored.
- **DFS only:** Frequently economical in memory and sometimes faster in practice, yet cannot claim path-length optimality. Again no side-by-side evidence would be generated.
- **Informed search (A*):** Potentially more efficient when a reliable heuristic is available, but requires heuristic design, admissibility verification and extra testing — work that lay outside the planned scope of the early SLEs.

4. Rationale / Justification

The primary justification is the numerical evidence collected in SLE-2 on the identical graph:

Scenario	BFS time (ms)	DFS time (ms)	BFS nodes	DFS nodes
Best-case (Goal B)	0.019620	0.004310	2	2
Average-case (Goal I)	0.020910	0.013620	9	6
Worst-case (Goal K)	0.126880	0.022080	11	11

In every scenario DFS recorded the lower average latency. Node expansion was identical for the extreme cases and strictly lower for DFS on the average-case goal (6 versus 9). Complementary py-spy flame graphs (BFS: 109 samples, DFS: 141 samples, both error-free) supplied a qualitative picture of where CPU samples accumulated, reinforcing the quantitative ranking without displacing it.

These measurements are bound to the particular graph, node ordering and goal placement that were used. A different topology could reverse the ranking; therefore every performance claim must be qualified. The architectural argument is independent of the numbers: the C4 Component layer already treated search as a replaceable concern. Introducing a second algorithm therefore required only local change inside that component, not a redesign of Input or Output. The decision consequently rests on two independent pillars — measurable behavioural differences that repay comparative study, and an architecture that makes dual implementation inexpensive to maintain and to test.

Keeping both algorithms also maximises learning value. The student can demonstrate when path optimality is the dominant requirement (select BFS) and when rapid exploration or modest peak memory is preferable (DFS may win on the graphs examined).

5. Consequences

Positive outcomes

- Reproducible, controlled comparison under identical graph, start node and goal set.
- Architecture stays coherent: search remains a swappable component, so both algorithms fit naturally.
- Concrete insight into the classic optimality-versus-efficiency trade-off is obtained.

Trade-offs accepted

- Two implementations must be kept correct and consistent with a shared interface.
- BFS may allocate a larger frontier; DFS may return a longer path.
- Observed timings are not universal; every claim must be qualified by the graph under test.

6. References

- Repository: https://github.com/juishinde136-arch/IAI_SLE2_BFS_DFS
- SLE-2 Performance Profiling Report (timeit data, node counts, py-spy visualisations) and SLE-3 C4 diagrams.

PART B — Summary of the SLE Journey

SLE-1 delivered working BFS and DFS code, a README and an AI-contribution log. SLE-2 shifted the focus from implementation to measurement: timeit averages, explicit node-expansion counters and py-spy flame graphs for three graded goals. SLE-3 lifted the concrete code into C4 views (Context, Container, Component, Code), exposing a clean separation among input, search and output. SLE-4 now formalises the central design choice — retain both algorithms — and anchors that choice in the measured numbers and the modular architecture already constructed.

The sequence therefore links four complementary skills: writing correct search code, evaluating performance rigorously, designing a maintainable system structure, and articulating a reasoned technical decision in both written and oral form.

PART C — AI Contribution Note

Tools employed: Large language models used strictly as drafting and organisational support.
Assistance received: Clarification of the ADR structure required by the course guideline, polishing of prose, and generation of candidate viva prompts.
Student ownership: Every experimental run was performed locally. All timing figures and node counts originate from those runs. Architectural statements were verified against the student's own SLE-3 diagrams. In the viva the student will answer from personal understanding and will immediately correct any statement that diverges from the actual artefacts.

PART D — Viva Preparation Sheet

1. **BFS in one sentence:** Explores the graph level by level with a queue (FIFO) and yields a shortest path by edge count on unweighted graphs.
2. **DFS in one sentence:** Pushes deep along one branch before backtracking (stack or recursion); path length is not guaranteed to be minimal.
3. **Headline experimental result:** DFS was faster on every tested goal (0.004310 ms / 0.013620 ms / 0.022080 ms) while expanding equal or fewer nodes.
4. **Why keep both algorithms?** Measured differences justify comparative study; the C4 Component boundary already made dual support low-cost.
5. **C4 levels (quick recall):** Context → Container → Component → Code. Search sits cleanly inside the Component layer.
6. **Scaling concern:** Larger graphs inflate BFS frontier memory and may exhaust DFS recursion depth; re-measure under the same three-goal protocol.
7. **Measurement method:** timeit (10 runs → min / avg / max) plus explicit node counters; identical graph and start node for both algorithms.
8. **Core trade-off:** BFS buys path optimality at possible memory cost; DFS may sacrifice optimality for speed and lower peak memory on the graphs examined.
9. **AI role (honest answer):** Helped with document organisation; all numbers and design claims were verified by the student against original artefacts.
10. **What if faculty asks "why not only the faster one?":** Because the ranking is graph-dependent; retaining both preserves the ability to choose optimality when it matters and speed when it does not.
11. **30-second decision pitch:** "I kept both BFS and DFS because my own profiling showed clear runtime and node-expansion differences, and the C4 architecture already isolated search as a swappable component, making dual implementation straightforward to test and explain."

Pre-submission & Pre-viva Checklist

- Timing and node figures match the final SLE-2 report exactly.
- ADR describes a decision that was genuinely taken.
- Filename follows faculty/Moodle convention (e.g. SLE4_25UAM132.docx).
- Document length is within the permitted 4-page limit.
- Ready to explain every number and design choice without reading from the page.